MACHINE LEARNING WITH SCIKIT-LEARN

Feature Engineering & Pipelines

03

Turn clean columns into model-ready numbers — and chain everything into leak-proof scikit-learn Pipelines.

Instructor: Shutchon Premchaisawatt

Master's Degree Program · Hands-on with Google Colab

AGENDA

Today's agenda

01 **Why models need numbers & scales**
the fare-vs-age problem

02 **Scaling**
Standard · MinMax · Robust

03 **Encoding categories**
OneHot vs Ordinal

04 **Feature engineering recipes**
bin, log, combine — then MEASURE

05 **Pipeline**
chain preprocessing + model

06 **ColumnTransformer**
different steps per column type

07 **Data leakage**
the silent score killer

FROM LAST SESSION

Clean data is not model food (yet)

raw data
S02 arrival

→

clean data
no missing, no dupes

→

transformed
numbers, sane scales — TODAY

→

model-ready
inside one Pipeline

Still text

sex = "female"
embarked = "S"
Models: TypeError.

Wildly different scales

age 0–80 vs fare 0–512
Distance-based models quietly break.

Manual transforms

Applying them by hand, twice (train/test),
invites leakage. Enter Pipelines.

KNN is blind when one column shouts

0.67

KNN accuracy
fare & age unscaled

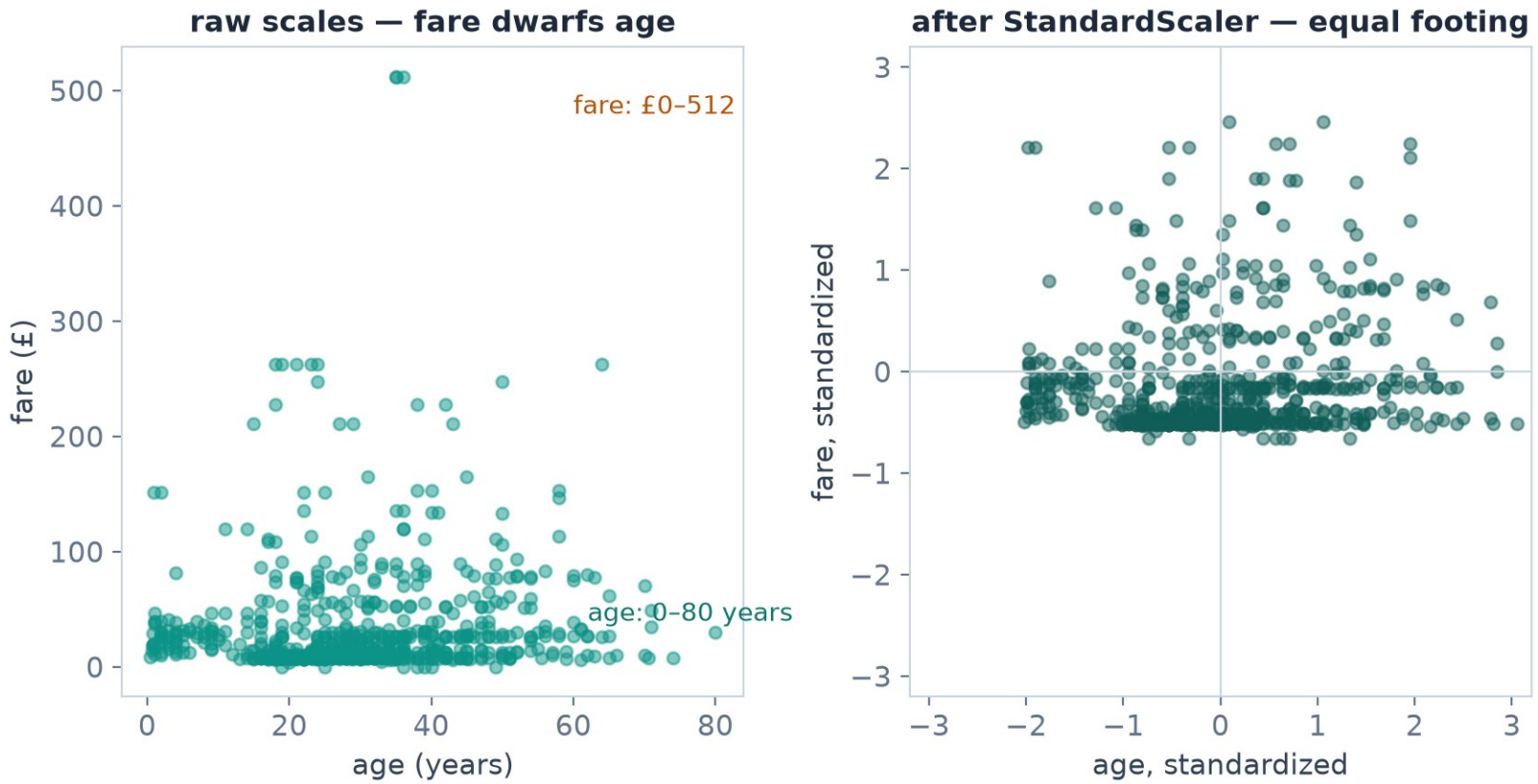
0.82

same KNN, same data
after StandardScaler

- **Why: KNN measures distance**
- Δfare can be 500 · Δage at most 80
- fare drowns every other signal
- Same story for SVM, K-Means, PCA
- **+15 points for free — just rescaling**

Trees (DecisionTree, RandomForest) split one feature at a time — scaling changes nothing for them.

Scaling, visually — before & after



- **Left: raw Titanic age × fare**
- fare's 0–512 range crushes the picture
- **Right: after StandardScaler**
- both features: mean 0, spread ±2 — equal footing
- **Same data, same KNN: 0.67 → 0.82**

SCALING

Three scalers you'll actually use

scaler	formula (idea)	use when
StandardScaler	center to mean 0, std 1	default first choice
MinMaxScaler	squash to [0, 1]	need a bounded range (NN, images)
RobustScaler	center by MEDIAN, scale by IQR	heavy outliers (fare!)

- **fit_transform on TRAIN**
- **transform on TEST**
- never fit on the test set!
  - (we'll automate this discipline with Pipelines in a few slides)

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train[num])   # learns train stats
X_test_s  = scaler.transform(X_test[num])       # reuses them
```

Who needs scaling? A field guide

Scale-sensitive

KNN — pure distance
SVM (RBF) — distance in kernel space
PCA — variance-based
Linear/Ridge/Lasso — coefficient penalties assume comparable scales
Neural nets — optimizers behave badly on wild scales

→ always scale these

Scale-indifferent

DecisionTree — thresholds per feature
RandomForest — many trees
GradientBoosting — trees inside

→ scaling optional (harmless, but no gain)

Practical rule: put scaling in the Pipeline anyway. It costs nothing and protects the model you swap in later.

Categories → numbers: two doors

ONE-HOT — one column per category



no fake order invented · grows wide with many categories

ORDINAL — one column, ordered ints



only when an order truly exists (size, rating, education)

- **Which door?**
- order exists → OrdinalEncoder
- no order → OneHotEncoder
- **sex, embarked → one-hot**
- **pclass → already ordered ints**
- Danger: encoding "female"=1 "male"=0 with a LINEAR model is fine, but "city": Bangkok=1, ChiangMai=2, Phuket=3 invents a fake order

OneHotEncoder in action

```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(handle_unknown="ignore")
oh.fit_transform(X_train[["sex", "embarked"]]).toarray()
```

sex_F	sex_M	emb_S	emb_C	emb_Q	…
1	0	1	0	0	
0	1	0	1	0	

- **handle_unknown="ignore"**
- at deploy time a NEW category appears → all-zeros row, no crash
- **Sparse output**
- fit_transform returns a sparse matrix (mostly zeros) — memory-friendly
  - 1000 categories? 1000 columns. High cardinality needs other tricks (group rare, hashing) — beyond today.

Recipes that earn their keep

Bin

age → is_child (age<12)
continuous → flag or band ("women and children first")

Log-transform

fare_log = log1p(fare)
tames skew 4.8 → ~0.5
helps linear & distance models

Combine

family_size = sibsp + parch + 1
lonely traveler vs big family
—
its own survival pattern

Ratio

fare_per_person = fare / family_size
prices normalized by group size
(fees, spend per tenure…)

```
X = df.drop(columns="survived")
X["family_size"] = X["sibsp"] + X["parch"] + 1
X["is_child"]    = (X["age"] < 12).astype(float)
X["fare_log"]    = np.log1p(X["fare"])
```

- **Then MEASURE**
- KNN 5-fold CV: 0.809 → 0.819 ✓ small win
- RandomForest: 0.807 → 0.801 ✗ no gain
- **Features must EARN their place — keep only what measurably helps**

THE PROBLEM

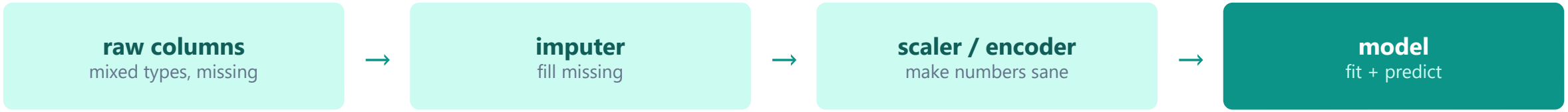
Doing it by hand — the bookkeeping trap

```
# manual way: 9 fiddly steps, every single time
imp_num = SimpleImputer(...).fit(X_train[num])
Xtr_num = imp_num.transform(X_train[num])
Xte_num = imp_num.transform(X_test[num])
scaler  = StandardScaler().fit(Xtr_num)
Xtr_num = scaler.transform(Xtr_num)
Xte_num = scaler.transform(Xte_num)
oh      = OneHotEncoder(...).fit(X_train[cat])
Xtr_cat = oh.transform(X_train[cat]).toarray()
Xte_cat = oh.transform(X_test[cat]).toarray()
X_train2 = np.hstack([Xtr_num, Xtr_cat])   # never again
```

- **Three failure modes:**
- ① forget a transform on test → crash or garbage
- ② accidentally fit on test → leakage
- ③ new data at deploy → re-do the whole dance
- **sklearn's answer: Pipeline + ColumnTransformer — next**

★ THE BIG IDEA

Pipeline: one object, all steps

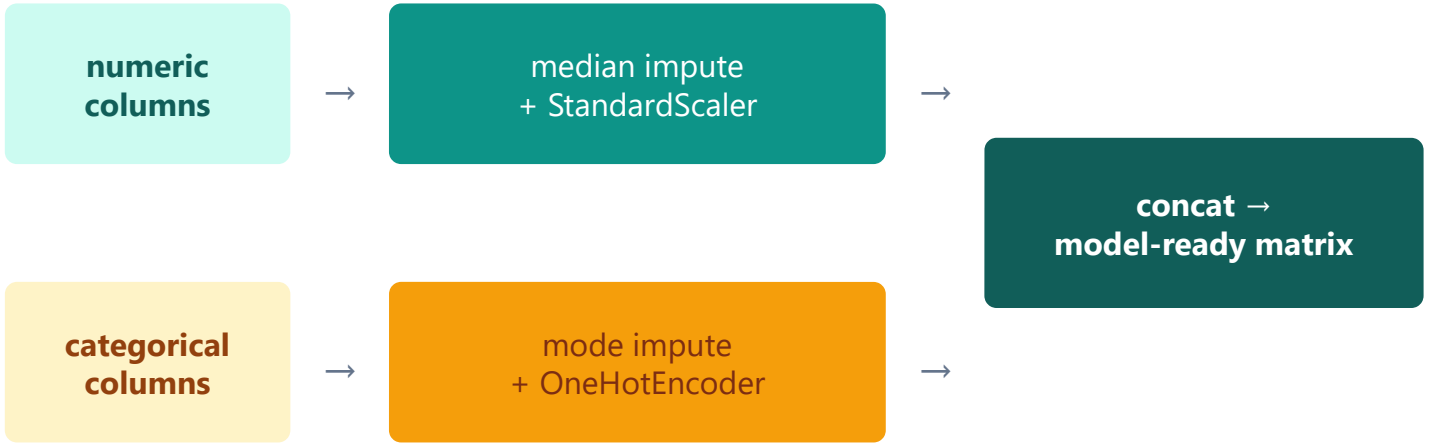



```
from sklearn.pipeline import Pipeline
pipe = Pipeline([("imp", SimpleImputer()),
                 ("sc",  StandardScaler())])
```

- **pipe.fit(X_train, y_train)**
- runs imp.fit_transform → sc.fit_transform in order
- **pipe.transform(X_test)**
- runs imp.transform → sc.transform (learned stats reused)
- **With a model inside: fit / predict / score — one call each**

★ THE BIG IDEA

ColumnTransformer: different food per column




```
pre = ColumnTransformer([
    ("num", num_pipe,        ["age","fare","sibsp","parch","pclass"]),
    ("cat", cat_pipe,        ["sex","embarked"])])
```

- Columns not listed? By default they pass through untouched (or set remainder="drop")

▸ **This one object replaces ALL the manual bookkeeping**

The 15-line template you'll reuse forever

```
num_pipe = Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc",  StandardScaler())])
cat_pipe = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oh",  OneHotEncoder(handle_unknown="ignore"))])
pre = ColumnTransformer([("num", num_pipe, num_cols),
                         ("cat", cat_pipe, cat_cols)])

clf = Pipeline([("pre", pre),
                ("model", KNeighborsClassifier())])

clf.fit(X_train, y_train)
clf.score(X_test, y_test)          # 0.816 — everything inside
```

- **Swap the model**
- KNeighborsClassifier() →
- LogisticRegression() →
- RandomForestClassifier()
- **…the 15 lines don't change**
  - Save clf with joblib → load in production → feed raw rows. Same result.

GUARANTEE

Inside clf.fit(): train-only discipline for free

clf.fit(X_train, y_train)
the only fit() you call

→

pre fits on TRAIN only
median, std, categories

→

model learns
on transformed train

clf.predict(X_new)
raw new data

→

pre.transform()
reuses TRAIN stats

→

model.predict()
done

Why this matters

Every statistic (imputation values, scaling mean/std, one-hot columns) is learned from TRAINING data only, then merely applied to test/future data. That discipline is exactly what the manual way keeps getting wrong.

DATA LEAKAGE

When information from OUTSIDE the training set — usually from the future or from the target itself — sneaks into your model.

The score looks amazing. The deployed model is useless.

It is the #1 silent killer of real-world ML projects — and AI assistants generate it happily.

LEAKAGE

Watch a leak happen — "lucky_ticket"

We invent a column that secretly knows the answer (85% of the time) — like using next month's refund record to predict this month's churn.

0.804

LogisticRegression
honest features

0.860

same model + lucky_ticket
+5.6 pts of pure illusion

- **In real projects the leak is rarely this obvious**
- "days_since_last_complaint" for churn — filed AFTER cancelling?
- "total_billed" including the refund after churn
- **Ask of every column: WHEN was this value known?**

LEAKAGE

The five usual suspects

suspect	how it leaks	defense
Scaler / imputer fit on ALL rows	test statistics bleed into training	Pipeline (fit inside)
Target-derived feature	lucky_ticket, post-outcome fields	ask WHEN the value existed
Duplicates across the split	model examined the exam in advance	drop_duplicates first
Group-aware splits	same patient/user in train AND test	GroupSplit (advanced)
Tuning on the test set	repeated peeking = soft memorization	CV for tuning, test ONCE (S04)

Defense in one line: split FIRST, transform INSIDE a Pipeline, touch the test set exactly once.

PRODUCTION REALITY

Monday morning: a category you never trained on

```
# training data had embarked: S, C, Q
# ...then a passenger boards at "B" (new port)
clf.predict(pd.DataFrame({"embarked": ["B"], ...}))

# OneHotEncoder(handle_unknown="ignore")  -> all-zero row, no crash
# default (error)                          -> ValueError 💥
```

- **Models meet data you never saw**
- new category, new range, new missing column
- **handle_unknown="ignore" is your seatbelt**
- Also from S02: SimpleImputer learned on train fills future NaNs
  - Everything is consistent because it all lives in ONE fitted object

RECAP

The four golden rules of preparation

1 · Split FIRST

Before any statistic touches any column.

fit() must never see test rows — not even one.

2 · Transform INSIDE

Imputers, scalers, encoders live in the Pipeline — fitted on train, applied everywhere.

3 · Ask WHEN

Every feature must be knowable at prediction time.

Post-outcome data = leak.

4 · MEASURE

New feature or transform? Compare scores.

Keep only what earns its place.

Split first. Transform inside. Ask WHEN. Measure.

Tape these four to your monitor — they prevent 90% of real-world ML accidents.

LAB 03 · DO IT IN GOOGLE COLAB

สร้าง **Pipeline** กันโปร่งใส — **Titanic, leak-free**

YOUR MISSION (โจทย์ภาษาไทยใน Colab)

- แบ่ง train/test 80:20 (stratify=y, random_state=42) ก่อนทำอะไรอื่น
- สร้าง num_pipe + cat_pipe + ColumnTransformer + KNN ใน Pipeline เดียว
- เทียบคะแนนแบบใส่/ไม่ใส่ StandardScaler — ต่างกันเท่าไหร่?
- เพิ่ม family_size และ is_child แล้ววัดว่าดีขึ้นจริงไหม
- โจทย์ชวนคิด: สร้างคอลัมน์ lucky_ticket (ทอยเหรียญแฟน target) แล้วดูคะแนน บวม จากนั้นลบทิ้งและอธิบายว่าทำไม

Learning goal

Build the full ColumnTransformer + Pipeline stack on Titanic, prove scaling matters, and feel a leakage illusion first-hand.

Vibe coding corner

ลองถาม AI: “เขียน ColumnTransformer ที่คอลัมน์ตัวเลขใช้ median+StandardScaler แต่คอลัมน์ category ใช้ most_frequent+OneHot” แล้วเช็กว่ามัน fit บน train เท่านั้นจริงไหม — นี่คือจุดที่ AI มักพลาด

KEY TAKEAWAYS

1. Scale-sensitive models (KNN, SVM, PCA) need comparable feature ranges — Titanic KNN jumped 0.67 → 0.82.
2. Encode categories honestly: one-hot when unordered, ordinal only when order truly exists.
3. Engineer features (bin, log, combine) — then MEASURE; they must earn their place.
4. ColumnTransformer + Pipeline = one object, train-only statistics, model-swappable, deploy-ready.
5. Leakage = future/target information in features. Split first, transform inside, ask WHEN each value existed.

NEXT

Session 04 — Model Evaluation & Cross-Validation — can you trust that number?